

✓ EE4410 - Cyber Security of Power Grids 🔒 ⚡

Lab 3 🧪 - Cyber-Physical Data Correlation 📊 📈

Dataset description

Physical

- IEEE 9-bus dynamic model is implemented as the physical power system using the Real-Time Digital Simulator (RTDS). The topology of the system is shown below.
- Voltages are saved from EMT simulations and capture the physical response to two different events:
 1. event a
 2. event b
- The loading of the power system is fixed and any change in its operating condition is the result of an event.

Cyber

- Communication network traffic is captured from a digital substation, utilizing a Hardware-in-the-Loop (HIL) testbed
- Traffic originates from **real protection IEDs** that are connected through a network switch to the RTDS
- GOOSE traffic throughput (packets/100 ms) is captured using Wireshark.

Dataset explanation

The datasets contain power system measurements and GOOSE traffic throughput, captured from one substation. Columns are the following:

- **For the power system measurements**
 1. Timestamp: Time in seconds
 2. Voltage magnitudes in per unit
- **For the OT network traffic throughput**
 1. Timestamp: Time in seconds
 2. Network traffic throughput - number of packets per 100 ms

✓ Student Details

Name: Zubair Al-Zubi

ID/number: 5342007

Import libraries

```
1 # general imports
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import random
6 from random import randint
7 import warnings
8 import seaborn as sns
9
10 # stats model imports
11 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
12 import statsmodels.api as sm
13 from statsmodels.tsa.arima.model import ARIMA
14 # suppress warning messages
15 warnings.filterwarnings('ignore')
16 sns.set_theme()
```

✓ Part 1: Familiarization task

Load the dataset

```
1 # Read csv files
2 normal_traffic = pd.read_csv("datasets/normal.csv")
3 eventa_traffic = pd.read_csv("datasets/event_A_traffic.csv")
4 eventb_traffic = pd.read_csv("datasets/event_B_traffic.csv")
5 eventa_volt=pd.read_csv("datasets/event_A_volt.csv")
6 eventb_volt=pd.read_csv("datasets/event_B_volt.csv")
```

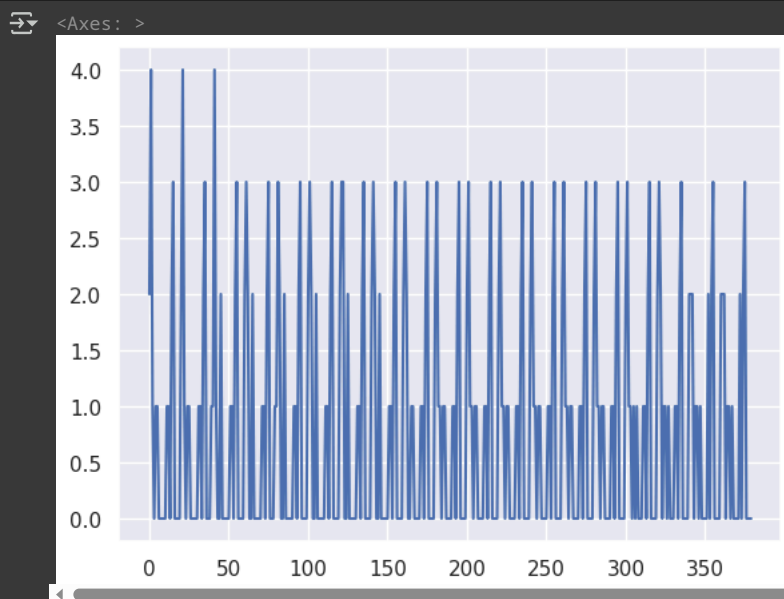
```
1 normal_traffic['numpckt'].describe()
```

↗

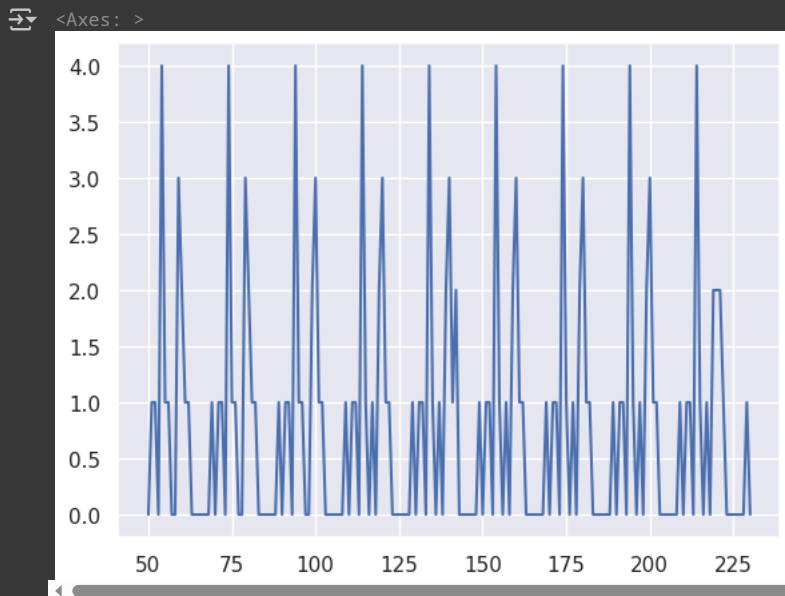
	numpckt
count	380.000000
mean	0.805263
std	1.037031
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	4.000000

dtype: float64

```
1 normal_traffic['numpckt'].plot()
2 # plt.savefig('plots/part-1/normal_traffic.svg', format='svg')
```

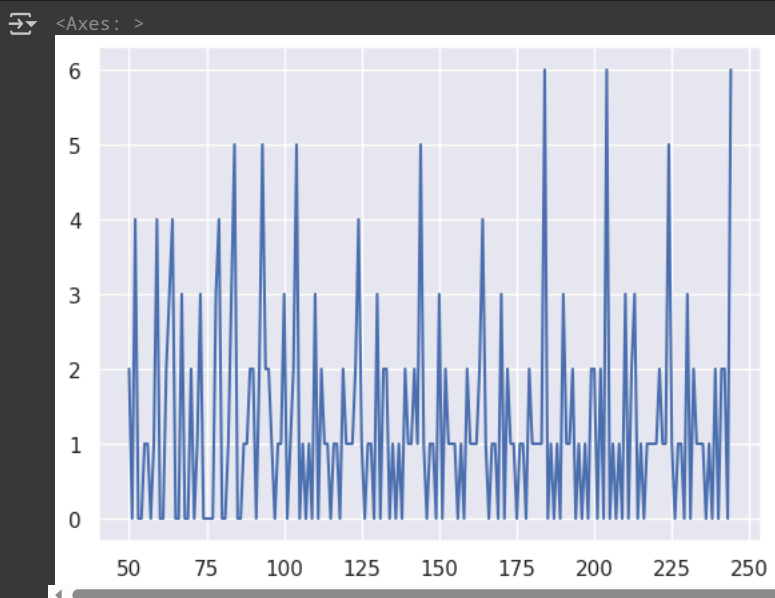


```
1 eventa_traffic['numpckt'][50:].plot()
2 # plt.savefig('plots/part-1/event_a_traffic.svg', format='svg')
```



- visualize the signals in the dataset and see what they represent?
- plot similar visualizations and see how the voltages and event traffic connect with each other

```
1 eventb_traffic['numpckt'][50:].plot()
2 # plt.savefig('plots/part-1/event_b_traffic.svg', format='svg')
```



Traffic plots

The GOOSE traffic signal `numpckt` data has been plotted for three different scenerios: normal operation, Event A, and Event B.

- Under normal operation: the signal shows regular and sparse activity, mostly ranging between 0 and 3 packets per 100ms.
- Under Event A: the traffic signal shows a structured but slightly denser pattern than normal operation, with bursts reaching up to 4 packets per 100ms.
- Under Event B: the signal shows an irregular and bursty pattern, with packet rates reaching up to 6 packets per 100ms.

✓ Voltage plots

The voltages at buses 5 to 9 during Event A and Event B have also been plotted below.

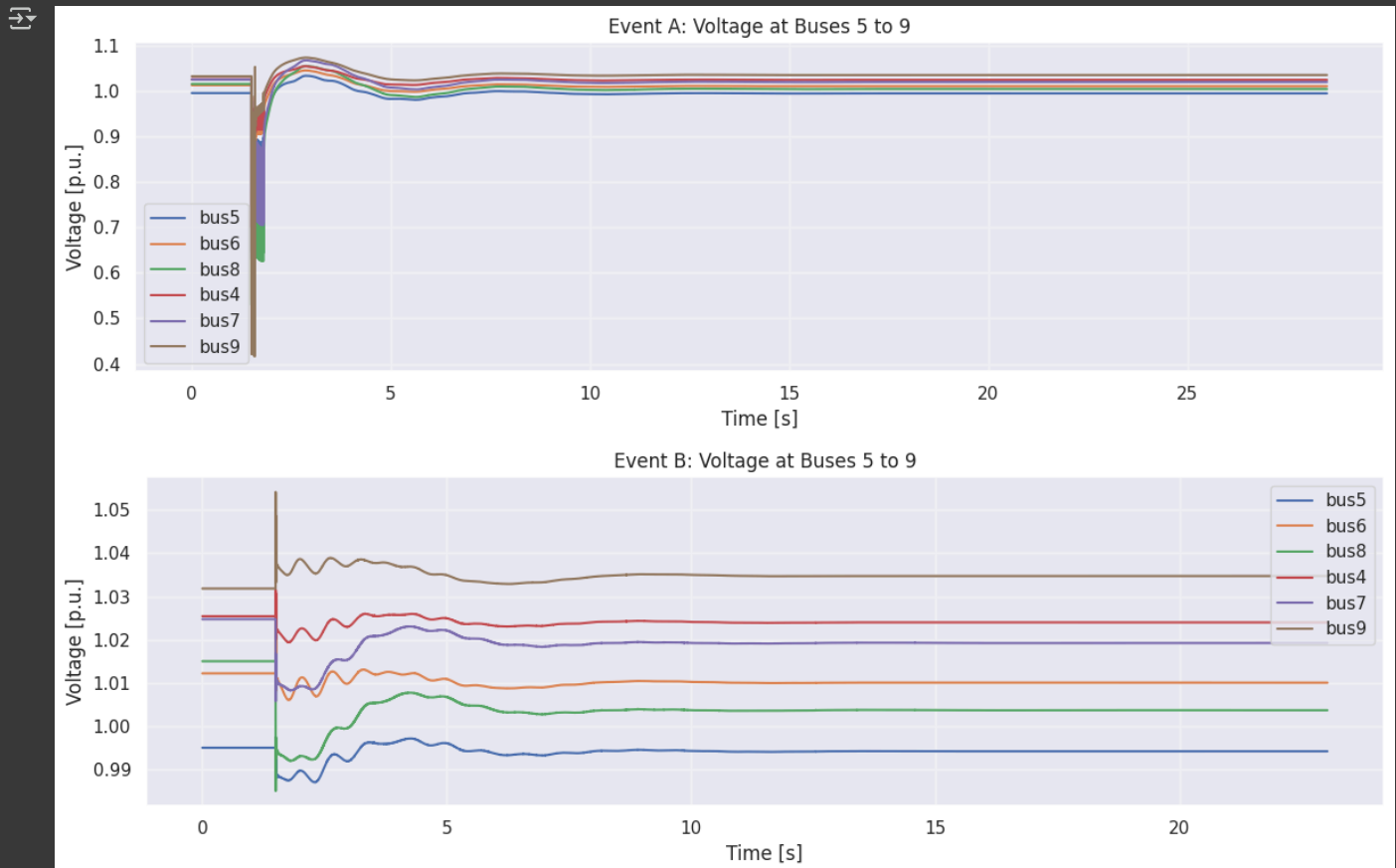
- In Event A, there was a sharp and significant drop in voltages across all buses (down to ~ 0.4 p.u.) at approximately $t = 2$ s, followed by quick recovery and stabilization around $t = 5$ -6s. This may be due to a short-circuit followed by successful breaker operation.
- In Event B, the plot shows a mild but noticeable deviation in voltage beginning around $t = 1$ s, followed by a series of oscillations before stabilizing. This could be due to a minor switching event rather than a physical fault.

```
1 eventa_volt["Time"] = eventa_volt["Time"].astype(float)
2 eventb_volt["Time"] = eventb_volt["Time"].astype(float)
```

```

3
4 # Plot Event A voltage across all buses
5 plt.figure(figsize=(12, 4))
6 for col in eventa_volt.columns[1:]:
7     plt.plot(eventa_volt["Time"], eventa_volt[col], label=col)
8 plt.title("Event A: Voltage at Buses 5 to 9")
9 plt.xlabel("Time [s]")
10 plt.ylabel("Voltage [p.u.]")
11 plt.legend()
12 plt.grid(alpha=0.3)
13 plt.tight_layout()
14 # plt.savefig('plots/part-1/event_a_voltage.svg', format='svg')
15 plt.show()
16
17 # Plot Event B voltage across all buses
18 plt.figure(figsize=(12, 4))
19 for col in eventb_volt.columns[1:]:
20     plt.plot(eventb_volt["Time"], eventb_volt[col], label=col)
21 plt.title("Event B: Voltage at Buses 5 to 9")
22 plt.xlabel("Time [s]")
23 plt.ylabel("Voltage [p.u.]")
24 plt.legend()
25 plt.grid(alpha=0.3)
26 plt.tight_layout()
27 # plt.savefig('plots/part-1/event_b_voltage.svg', format='svg')
28 plt.show()
29

```



✓ Part 2: Implement correlation techniques/methods

For this lab, it is recommended to use the python [statsmodel](#) library.

✓ ARMA model

Model contains p Auto-Regressive (AR) terms and q Moving-Average (MA) terms. The model is given by:

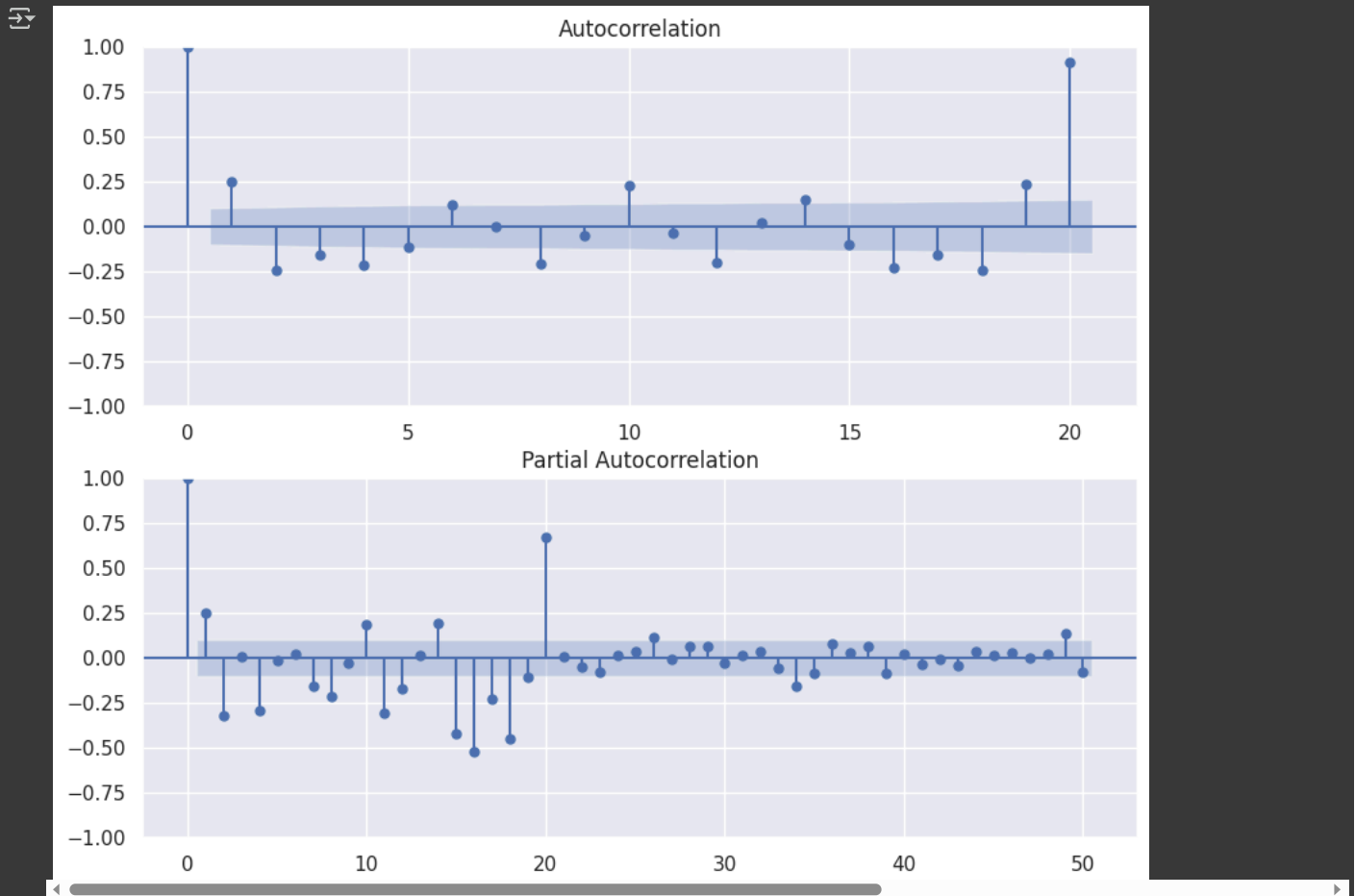
$$X_t = \sum_{i=1}^p \varphi_i X_{t-i} + \sum_{i=1}^q \theta_i \varepsilon_{t-i} + \varepsilon_t$$

hint: to find the right values for p, q , test for autocorrelation (MA part) and partial-autocorrelation (for the AR part).

✓ ACF and PACF to find optimal p, q

The PACF covers the AR and ACF covers the MA part

```
1 fig = plt.figure(figsize=(10,8))
2 ax1 = fig.add_subplot(211)
3 fig = sm.graphics.tsa.plot_acf(normal_traffic['numpckt'].dropna(),lags=20,ax=ax1)
4
5 ax2 = fig.add_subplot(212)
6 fig = sm.graphics.tsa.plot_pacf(normal_traffic['numpckt'].dropna(),lags=50,ax=ax2)
```



✓ Train ARMA/ARIMA model

```
1 train_end=200
2 training = normal_traffic['numpckt'][0:train_end]
3 testing = normal_traffic['numpckt'][train_end+1:]
```

- reference to ARIMA function from statsmodel library:
<https://www.statsmodels.org/stable/generated/statsmodels.tsa.arima.model.ARIMA.html>
- example ARMA implementation: https://www.statsmodels.org/stable/examples/notebooks/generated/tsa_arma_1.html

```
1 # this is an example with some random values of p,q to highlight the implementation.
2 # modify this part with your p,q values.
3 p= 1
4 q= 2
5 # here we define the ARMA model to be trained on our training samples.
6 # This is an ARMA(p,q) model
7 model=ARIMA(training,order=(p,0,q)) # p, d, q. we set d=0 since there is no differencing to be applied.
8 # train the model on the training dataset
```

```

9 model_fit=model.fit()
10 print(model_fit.summary())

```



SARIMAX Results

```

=====
Dep. Variable:          numpckt      No. Observations:          200
Model:                ARIMA(1, 0, 2)  Log Likelihood            -260.760
Date:                  Sat, 31 May 2025  AIC                        531.520
Time:                  23:34:23        BIC                        548.011
Sample:                0              HQIC                       538.194
                                - 200
Covariance Type:        opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
const	0.8047	0.004	179.171	0.000	0.796	0.814
ar.L1	0.5564	0.088	6.356	0.000	0.385	0.728
ma.L1	-0.3920	13.768	-0.028	0.977	-27.377	26.593
ma.L2	-0.6080	8.352	-0.073	0.942	-16.977	15.761
sigma2	0.7780	10.704	0.073	0.942	-20.201	21.757

```

=====
Ljung-Box (L1) (Q):                0.36  Jarque-Bera (JB):                17.89
Prob(Q):                            0.55  Prob(JB):                      0.00
Heteroskedasticity (H):              0.73  Skew:                          0.73
Prob(H) (two-sided):                 0.21  Kurtosis:                      2.86
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Forecast/predict using ARMA

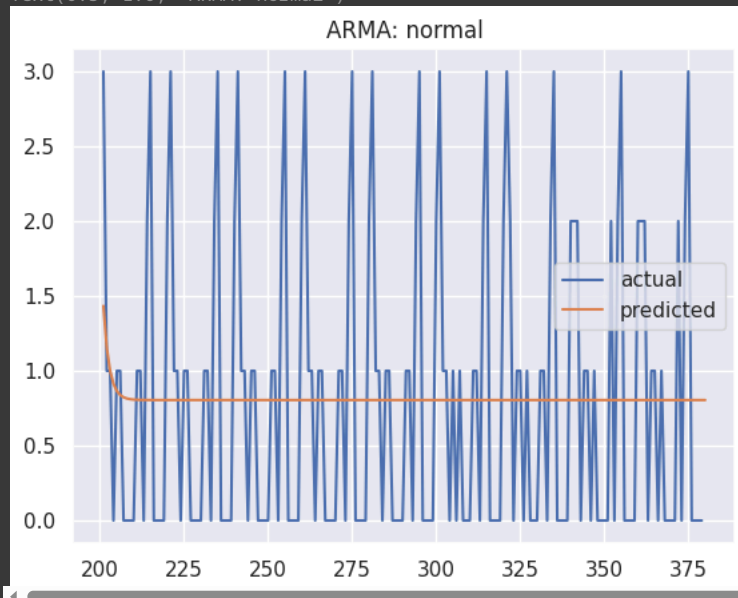
```

1 t1=train_end+1
2 t2= len(normal_traffic['numpckt'])
3 # here we predict the remaining samples using the trained ARMA model
4 forecast=model_fit.predict(start=t1,end=t2, dynamic=True)
5 # plot the actual signal
6 testing.plot()
7 # plot the predicted values
8 forecast.plot()
9 plt.legend(['actual','predicted'])
10 plt.title('ARMA: normal')
11 # plt.savefig('plots/part-2/normal_arma.svg', format='svg')

```



Text(0.5, 1.0, 'ARMA: normal')



```

1 # here we plot the residuals, i.e., differences between the actual and predicted values
2 # ideally, the residual ~0
3
4 # plot the residuals/errors
5 residuals = testing- forecast
6 plt.figure(figsize=(10,4))
7 plt.plot(residuals)
8 plt.axhline(0, linestyle='--', color='k')
9 plt.title('Residuals from ARMA Model', fontsize=20)

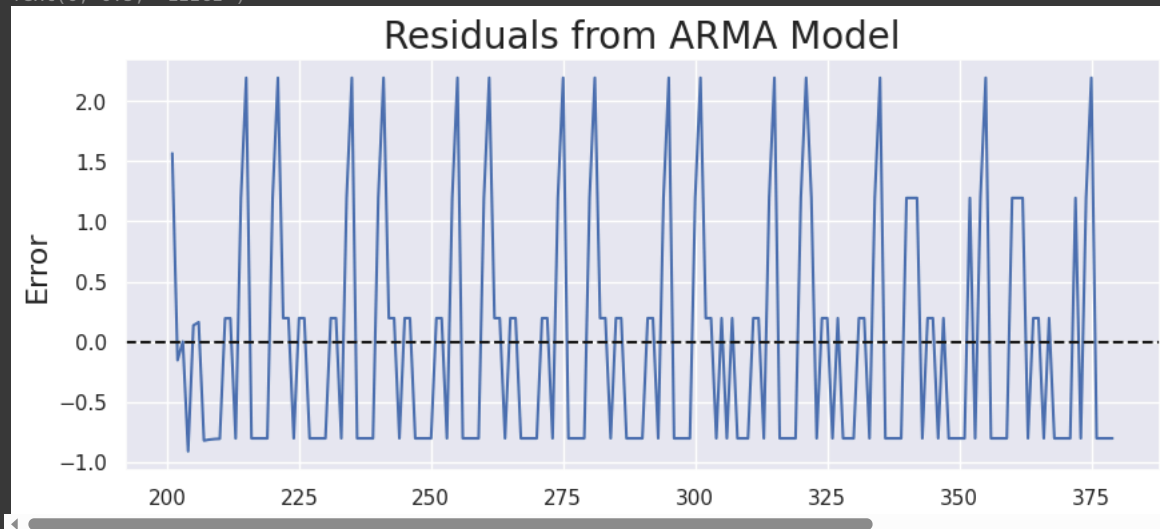
```

```

10 plt.ylabel('Error', fontsize=16)
11 # plt.savefig('plots/part-2/normal_residuals.svg', format='svg')

```

↔ Text(0, 0.5, 'Error')



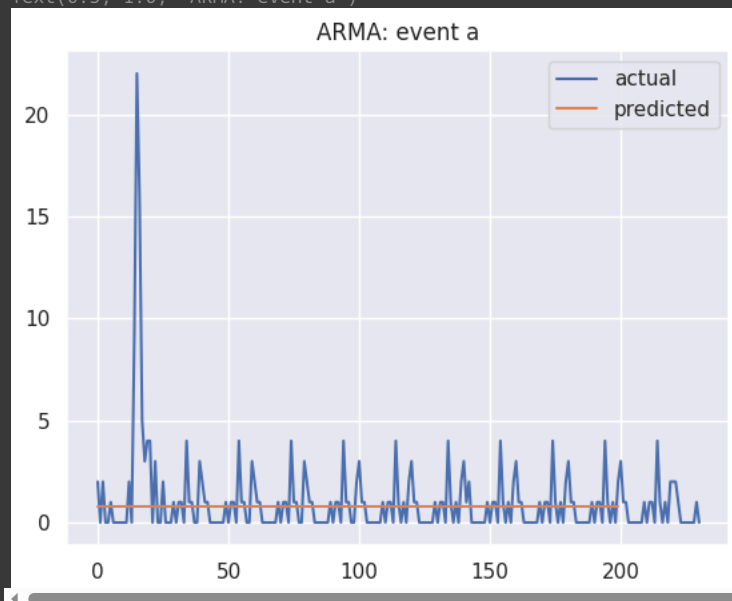
using the trained ARMA model, we repeat only the predictions for events a and b and plot the corresponding residuals as shown below

```

1 # comparison of predicted values vs entire series 'event a traffic'
2 forecast_a=model_fit.predict(dynamic=True)
3 eventa_traffic['numpckt'].plot()
4 forecast_a.plot()
5 plt.legend(['actual','predicted'])
6 plt.title('ARMA: event a')
7 # plt.savefig('plots/part-2/event_a_arma.svg', format='svg')

```

↔ Text(0.5, 1.0, 'ARMA: event a')

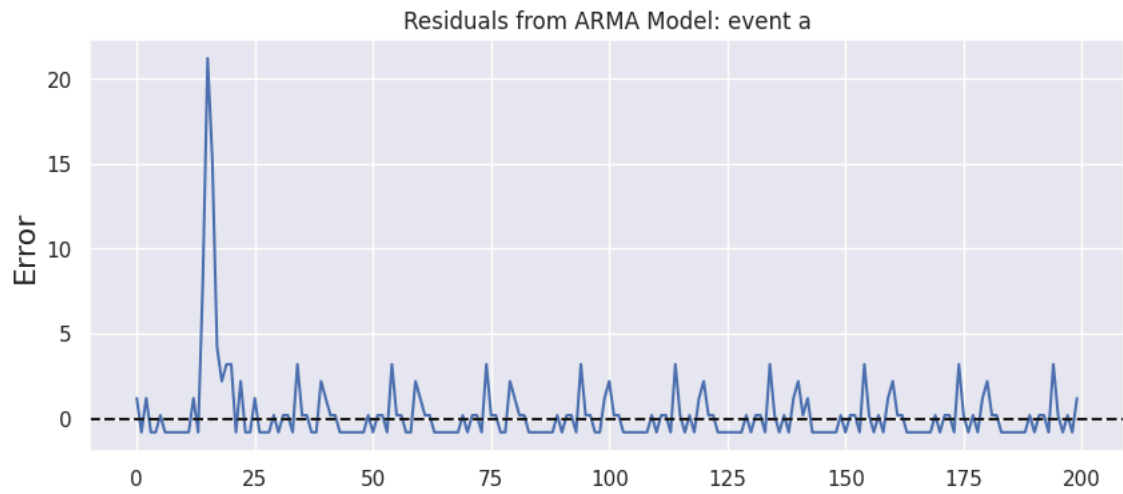


```

1 residuals = eventa_traffic['numpckt']- forecast_a
2 plt.figure(figsize=(10,4))
3 plt.plot(residuals)
4 plt.axhline(0, linestyle='--', color='k')
5 plt.title('Residuals from ARMA Model: event a', fontsize=12)
6 plt.ylabel('Error', fontsize=16)
7 # plt.savefig('plots/part-2/event_a_residuals.svg', format='svg')

```

Text(0, 0.5, 'Error')

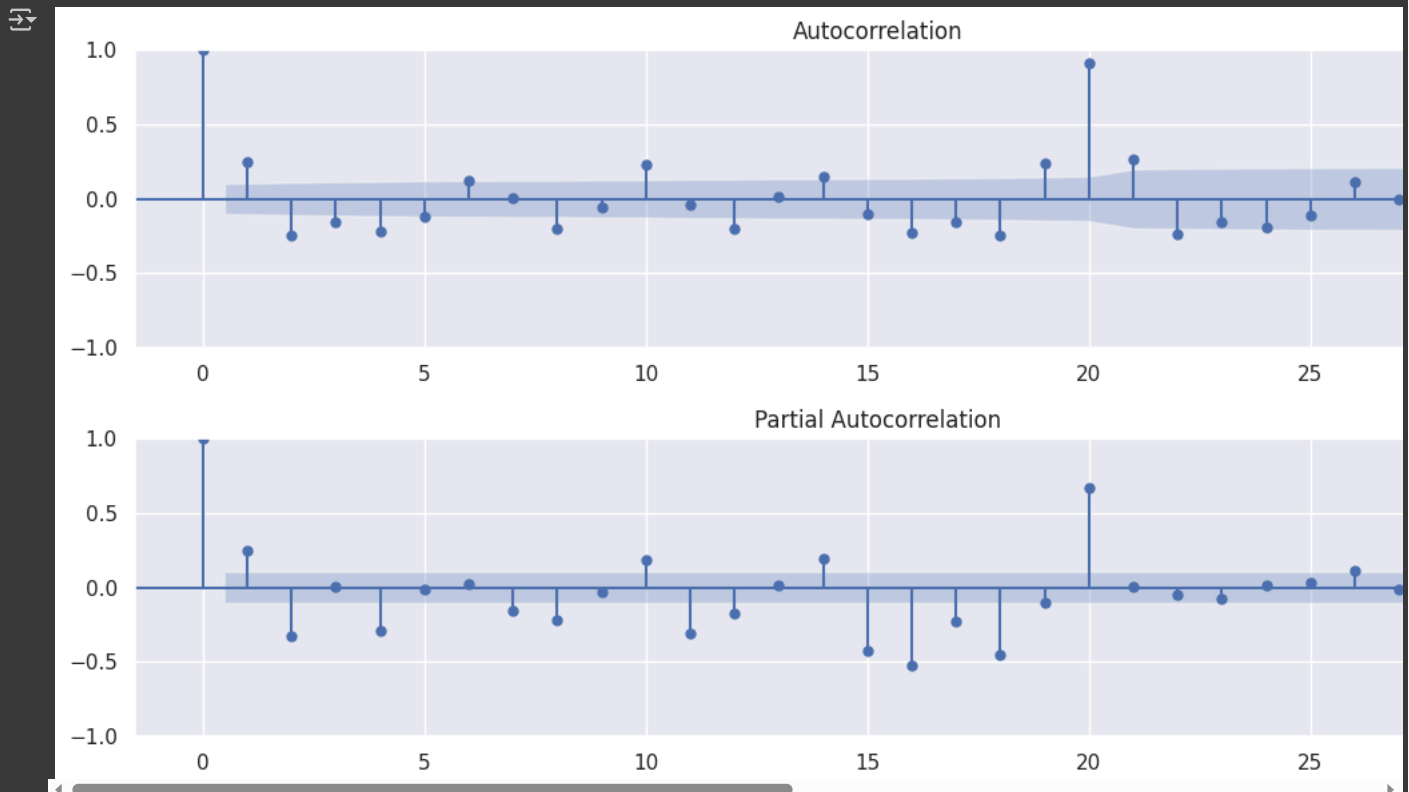


please repeat for event b

Event B

Find Optimal (p, q) using ACF/PACF

```
1 plt.figure(figsize=(12, 6))
2
3 # ACF (MA part)
4 plt.subplot(211)
5 sm.graphics.tsa.plot_acf(normal_traffic['numpckt'].dropna(), lags=30, ax=plt.gca())
6
7 # PACF (AR part)
8 plt.subplot(212)
9 sm.graphics.tsa.plot_pacf(normal_traffic['numpckt'].dropna(), lags=30, ax=plt.gca())
10
11 plt.tight_layout()
12 plt.show()
```



Train ARMA model

```

1 from statsmodels.tsa.arima.model import ARIMA
2
3 # 80-20 split
4 train_end = int(len(normal_traffic) * 0.8)
5 training = normal_traffic['numpckt'][:train_end]
6 testing = normal_traffic['numpckt'][train_end:]
7
8 # Fit ARMA(p,d,q) where d = 0
9 model = ARIMA(training, order=(1, 0, 1))
10 model_fit = model.fit()
11
12 # Print summary
13 print(model_fit.summary())
14

```



SARIMAX Results

```

=====
Dep. Variable:          numpckt      No. Observations:          304
Model:                ARIMA(1, 0, 1)  Log Likelihood            -417.335
Date:                  Sat, 31 May 2025  AIC                        842.671
Time:                  23:36:47        BIC                        857.539
Sample:                0              HQIC                       848.618
                                - 304
Covariance Type:        opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          0.8158      0.100        8.129      0.000        0.619        1.013
ar.L1         -0.1883      0.099       -1.910      0.056       -0.381        0.005
ma.L1          0.6397      0.089        7.225      0.000        0.466        0.813
sigma2          0.9110      0.100        9.145      0.000        0.716        1.106
=====
Ljung-Box (L1) (Q):                0.21      Jarque-Bera (JB):                24.25
Prob(Q):                           0.65      Prob(JB):                  0.00
Heteroskedasticity (H):              0.73      Skew:                      0.66
Prob(H) (two-sided):                0.11      Kurtosis:                  2.60
=====

```

Warnings:

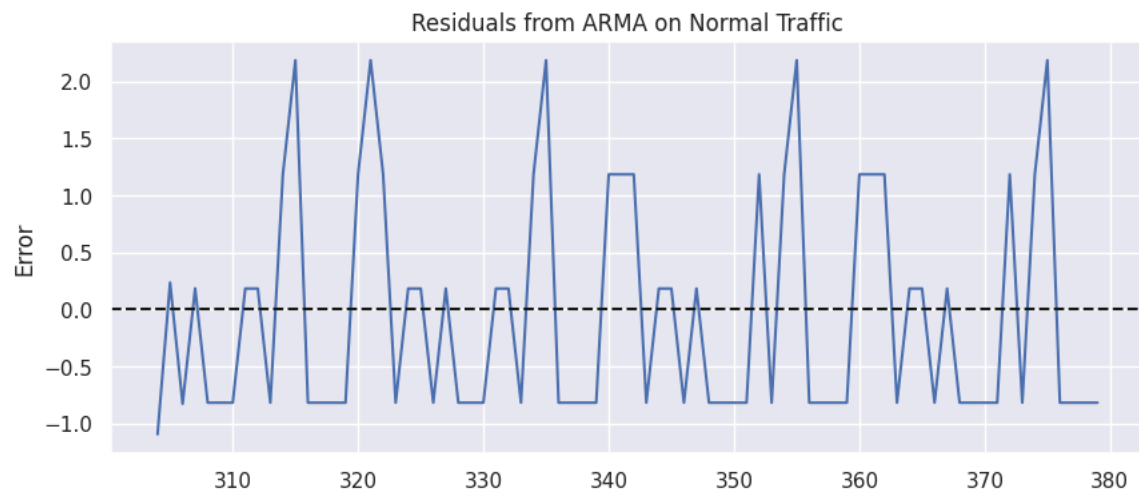
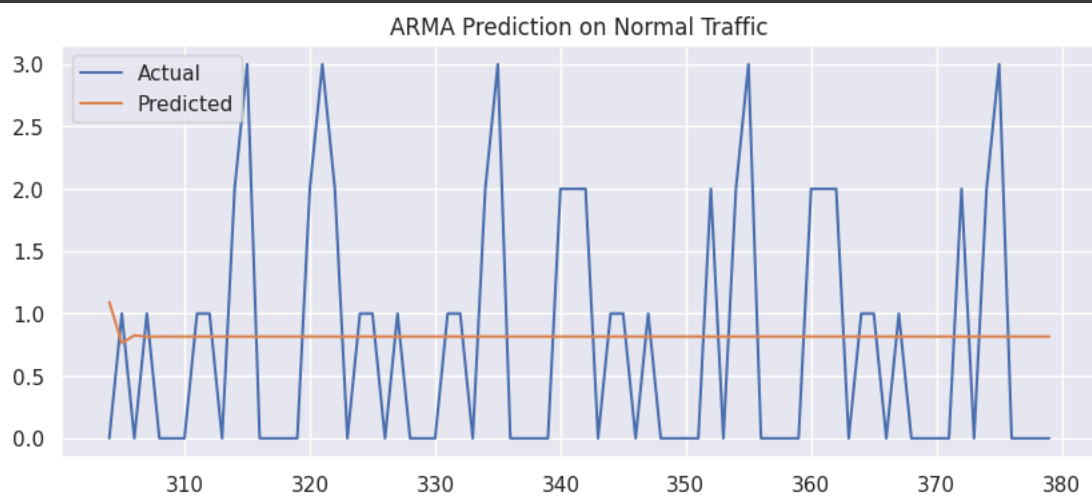
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

- Predict Normal Traffic & Plot Residuals

```

1 # Predict on test set
2 forecast = model_fit.predict(start=train_end, end=len(normal_traffic)-1, dynamic=True)
3
4 # Plot actual vs predicted
5 plt.figure(figsize=(10, 4))
6 testing.plot(label='Actual')
7 forecast.plot(label='Predicted')
8 plt.legend()
9 plt.title("ARMA Prediction on Normal Traffic")
10 plt.show()
11
12 # Plot residuals
13 residuals = testing - forecast
14 plt.figure(figsize=(10, 4))
15 plt.plot(residuals)
16 plt.axhline(0, color='k', linestyle='--')
17 plt.title("Residuals from ARMA on Normal Traffic")
18 plt.ylabel("Error")
19 plt.show()
20

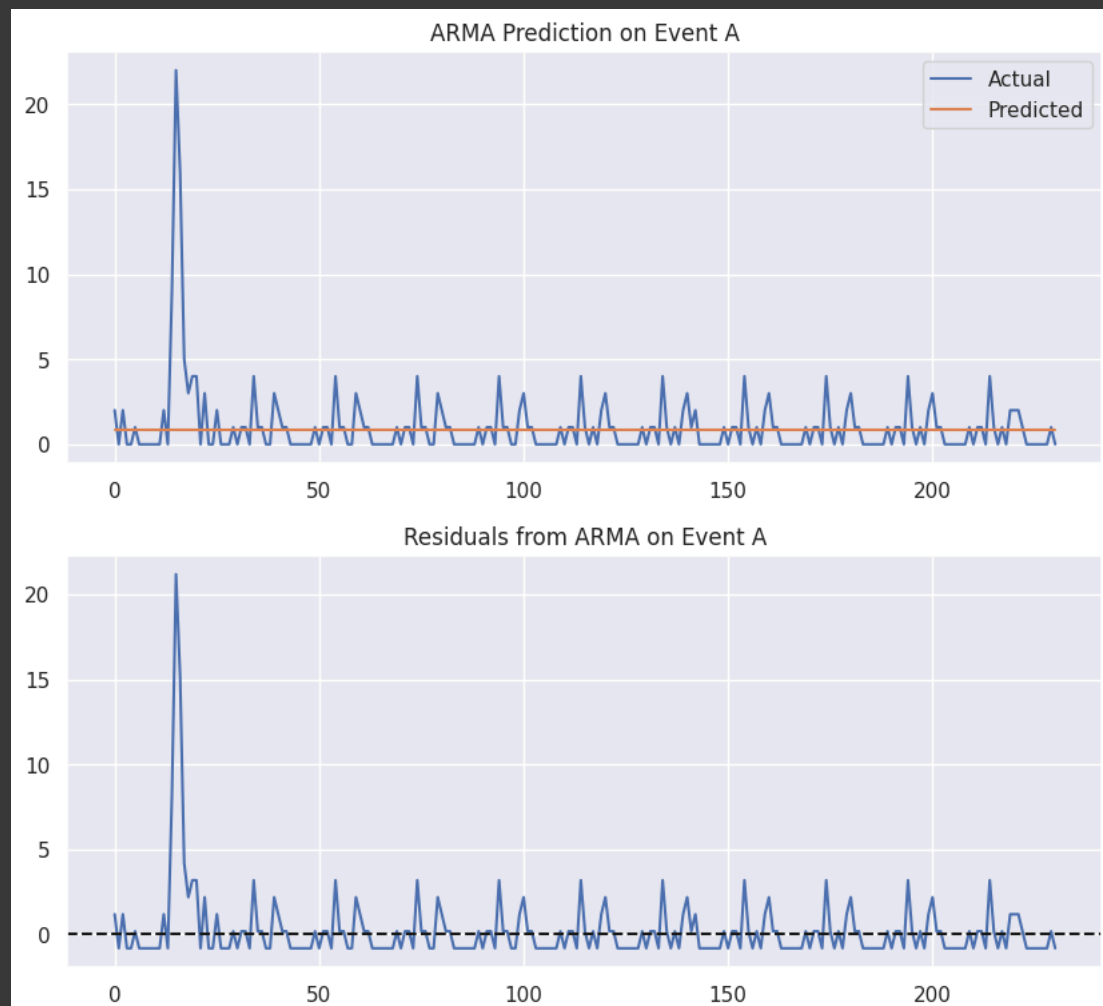
```



✓ Predict Events A and B & Plot Residuals

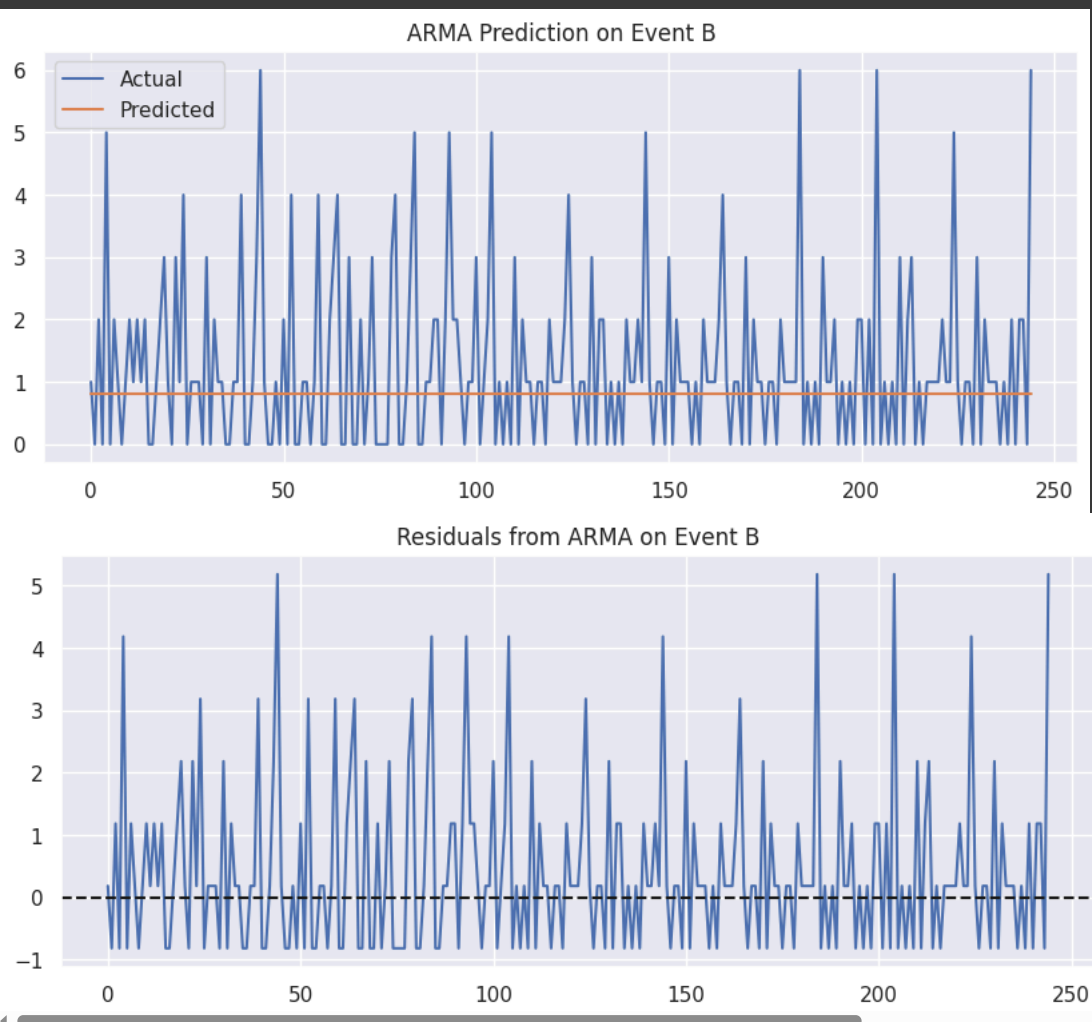
Event A

```
1 forecast_a = model_fit.predict(dynamic=True)
2 residuals_a = eventa_traffic['numpckt'] - forecast_a[:len(eventa_traffic)]
3
4 plt.figure(figsize=(10, 4))
5 eventa_traffic['numpckt'].plot(label="Actual")
6 forecast_a[:len(eventa_traffic)].plot(label="Predicted")
7 plt.legend()
8 plt.title("ARMA Prediction on Event A")
9 plt.show()
10
11 plt.figure(figsize=(10, 4))
12 plt.plot(residuals_a)
13 plt.axhline(0, color='k', linestyle='--')
14 plt.title("Residuals from ARMA on Event A")
15 plt.show()
16
```



Event B

```
1 forecast_b = model_fit.predict(dynamic=True)
2 residuals_b = eventb_traffic['numpckt'] - forecast_b[:len(eventb_traffic)]
3
4 plt.figure(figsize=(10, 4))
5 eventb_traffic['numpckt'].plot(label="Actual")
6 forecast_b[:len(eventb_traffic)].plot(label="Predicted")
7 plt.legend()
8 plt.title("ARMA Prediction on Event B")
9 plt.show()
10
11 plt.figure(figsize=(10, 4))
12 plt.plot(residuals_b)
13 plt.axhline(0, color='k', linestyle='--')
14 plt.title("Residuals from ARMA on Event B")
15 plt.show()
16
```



✓ Train SARMA model

the API reference of the SARMA class: <https://www.statsmodels.org/dev/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html>

```
1 # define the weights for the SARMA. These are just some example values to highlight implementation.
2 p=1
3 q=2
4 d=0
5
6 P=1
7 D=0
8 Q=2
9 s=4
10 # define and train the SARMA model
11 model=sm.tsa.statespace.SARIMAX(training, order=(p,d,q), seasonal_order=(P,D,Q,s))
12 results=model.fit()
```

```
1 # results from SARMA model
2 print(results.summary())
```



SARIMAX Results

```
=====
Dep. Variable:          numpckt      No. Observations:          200
Model:                SARIMAX(1, 0, 2)x(1, 0, 2, 4)      Log Likelihood          -266.439
Date:                  Sat, 31 May 2025                  AIC              546.879
Time:                  23:34:26                           BIC              569.967
Sample:                0                                HQIC              556.222
Covariance Type:      - 200
                        opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.5473	0.198	2.766	0.006	0.159	0.935
ma.L1	-0.3599	0.177	-2.038	0.042	-0.706	-0.014
ma.L2	-0.3593	0.068	-5.247	0.000	-0.493	-0.225
ar.S.L4	0.9983	0.001	734.848	0.000	0.996	1.001
ma.S.L4	-1.8866	0.161	-11.724	0.000	-2.202	-1.571
ma.S.L8	0.9757	0.169	5.757	0.000	0.643	1.308

```

sigma2      0.7037      0.109      6.461      0.000      0.490      0.917
=====
Ljung-Box (L1) (Q):      0.00      Jarque-Bera (JB):      0.38
Prob(Q):      1.00      Prob(JB):      0.83
Heteroskedasticity (H):      0.64      Skew:      0.06
Prob(H) (two-sided):      0.07      Kurtosis:      2.82
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

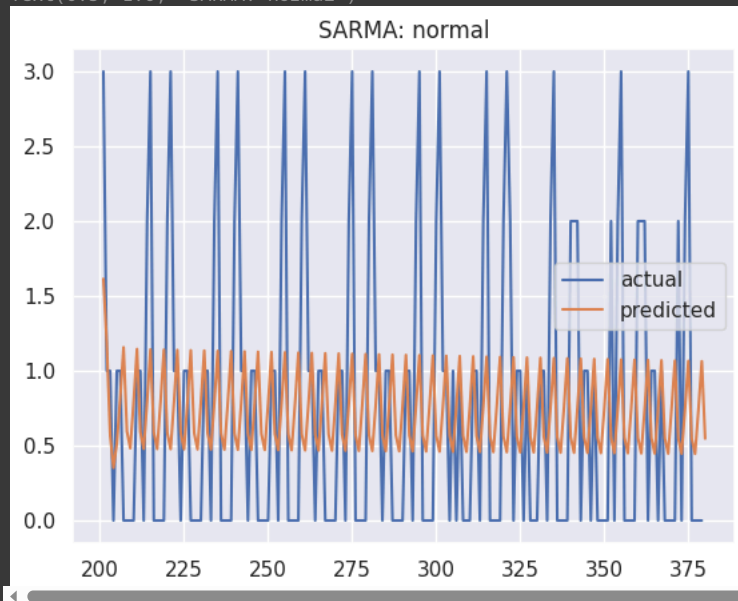
Forecast using SARMA

```

1 forecast_sarma=results.predict(start=t1,end=t2, dynamic=True)
2 # plot the actual signal
3 testing.plot()
4 # plot the predicted values
5 forecast_sarma.plot()
6 plt.legend(['actual','predicted'])
7 plt.title('SARMA: normal')

```

Text(0.5, 1.0, 'SARMA: normal')



```

1 # plot the residuals/errors
2 residuals = testing- forecast_sarma
3 plt.figure(figsize=(10,4))
4 plt.plot(residuals)
5 plt.axhline(0, linestyle='--', color='k')
6 plt.title('Residuals from SARMA Model', fontsize=20)

```